MANUAL DE PROGRAMADOR

Programa: Repaso general en C++

Actividad 6 · Laboratorio de Programación · 5.º año

Documento técnico para comprender la organización del código, las funciones y los comandos necesarios para compilar y probar la aplicación.

1. Objetivo y alcance

El programa reúne tres ejercicios de C++: recursividad con caso base, búsqueda secuencial en un arreglo y modificación de variables originales mediante punteros. El alcance es educativo y se limita a una aplicación de consola.

2. Estructura del proyecto

```
repasogeneral/
        .gitignore
        LICENSE
        README.md
    docs/
        manual_usuario_v1.0.0.pdf
        manual_programador_v1.0.0.pdf
    src/
        main.cpp
    capturas/
```

3. Bibliotecas y prototipos

```
#include <iostream>
using namespace std;
int sumaRecursiva(int n);
void intercambiarValores(int* ptrA, int* ptrB);
```

iostream habilita la entrada y salida estándar, incluida la salida con cout. Los prototipos anuncian las funciones antes de que sean llamadas desde main.

4. Reto 1: recursividad

```
int sumaRecursiva(int n) {
if (n <= 0) return 0;    return n + sumaRecursiva(n - 1);
}
```

La condición n <= 0 es el caso base y detiene las llamadas. En cada llamada, el argumento disminuye en uno. Para n = 5, el cálculo equivale a 5 + 4 + 3 + 2 + 1 + 0 = 15. Sin una condición de corte, las llamadas podrían continuar hasta provocar un desbordamiento de pila.

5. Reto 2: búsqueda secuencial

```
const int TAM = 10;
int vectorDatos[TAM] = {12, 45, 78, 90, 122, 5, 18, 33, 67, 99};
int numeroBuscar = 18;
bool hallado = false;

for (int i = 0; i < TAM; i++) {
  if (vectorDatos[i] == numeroBuscar) {
      cout << &vectorDatos[i] << endl;
```

```
        hallado = true;
        break;
           }
}
```

El ciclo revisa cada posición desde el índice 0 hasta TAM - 1. La comparación identifica el valor buscado. &vectorDatos[i] obtiene la dirección de memoria del elemento actual y break termina el ciclo cuando ya se encontró el valor. En el arreglo de ejemplo, el valor 18 ocupa el índice 6.

6. Reto 3: intercambio con punteros

```
void intercambiarValores(int* ptrA, int* ptrB) {
   int temporal = *ptrA;
   *ptrA = *ptrB;
    *ptrB = temporal;
}
int x = 100, y = 500;
 intercambiarValores(&x, &y);
```

Los parámetros int* reciben direcciones de memoria. En la llamada se utiliza &x y &y para enviar las direcciones de las variables originales. El operador * permite acceder a los valores guardados en esas direcciones. La variable temporal conserva el primer valor mientras se realiza el intercambio.

7. Compilación y ejecución

Desde PowerShell, en la carpeta raíz del proyecto:

```
g++ src/main.cpp -o src/repasogeneral.exe
.\src\repasogeneral.exe
```

El primer comando compila el archivo fuente y crea el ejecutable. El segundo lo inicia. Si se modifica el código, guardar el archivo y volver a compilar antes de ejecutarlo.

8. Pruebas básicas

Prueba	Resultado esperado
Suma recursiva con n = 5	15
Buscar el valor 18	Encontrado en índice 6; se muestra su dirección de memoria
Intercambiar X = 100 e Y = 500	X = 500 e Y = 100
Valor buscado ausente	Se informa que no existe en el arreglo si se cambia numeroBuscar por un valor

9. Control de versiones y documentación

La actividad pide subir el proyecto a GitHub y configurar un segundo destino en GitLab. Antes de enviar cambios, verificar los archivos incluidos, compilar el programa y guardar capturas de la ejecución. No se recomienda versionar archivos ejecutables; el archivo .gitignore puede excluir *.exe, .vscode/ y *.out.

10. Limitaciones

Las direcciones de memoria impresas pueden cambiar entre ejecuciones. El programa es demostrativo: no utiliza base de datos, interfaz gráfica ni conexión de red. Este manual describe el código de la actividad y no incorpora funciones que no estén contempladas en ese alcance.

Control de revisión

Versión	Fecha	Descripción
v1.0.0	2026	Primera versión del manual de programador.